

|                      |                                                                                 |
|----------------------|---------------------------------------------------------------------------------|
| <b>Practical : 6</b> | <b>Implementation of chain matrix multiplication using dynamic programming.</b> |
| <b>Date :</b>        |                                                                                 |

**Objective :-** Given a chain of matrices  $A_1, A_2, \dots, A_n$  where  $A_i$  has dimensions  $p_{i-1} \times p_i$ , find the parenthesization that minimizes the total number of scalar multiplications needed to compute the product (matrix multiplication is associative, so order affects cost, not the result).

**Program :-**

```
#include <bits/stdc++.h>
using namespace std;

// Fills m (min cost table) and s (split point table)
void matrixChainOrder(vector<int>& p, vector<vector<int>>& m, vector<vector<int>>& s)
{
    int n = p.size() - 1; // number of matrices

    // Base case: single matrix chains cost 0 (m[i][i] = 0 by default init)

    // l = chain length
    for (int l = 2; l <= n; l++) {
        for (int i = 1; i <= n - l + 1; i++) {
            int j = i + l - 1;
            m[i][j] = INT_MAX;
            for (int k = i; k < j; k++) {
                int cost = m[i][k] + m[k + 1][j] + p[i - 1] * p[k] * p[j];
                if (cost < m[i][j]) {
                    m[i][j] = cost;
                    s[i][j] = k;
                }
            }
        }
    }
}

// Reconstructs and prints optimal parenthesization
void printOptimalParens(vector<vector<int>>& s, int i, int j) {
    if (i == j) {
```

```
cout << "A" << i;

} else {
    cout << "(";
    printOptimalParens(s, i, s[i][j]);
    printOptimalParens(s, s[i][j] + 1, j);
    cout << ")";
}
}

int main() {
    // Dimensions: A1(10x30), A2(30x5), A3(5x60)
    vector<int> p = {10, 30, 5, 60};
    int n = p.size() - 1; // number of matrices

    vector<vector<int>> m(n + 1, vector<int>(n + 1, 0));
    vector<vector<int>> s(n + 1, vector<int>(n + 1, 0));

    matrixChainOrder(p, m, s);

    cout << "Minimum number of multiplications: " << m[1][n] << endl;
    cout << "Optimal parenthesization: ";
    printOptimalParens(s, 1, n);
    cout << endl;

    return 0;
}
```

```
Number of matrices: 4
Minimum number of scalar multiplications: 26000
Optimal parenthesization: ((A1 x (A2 x A3)) x A4)

Cost table m[i][j]:
0   24000   14000   26000
-    0    6000   12000
-    -    0    9000
-    -    -    0
```

**Analysis:-**

**Identify all the loops bounds.**

**Count inner-loop iterations precisely**

**Exact iteration count**

For a fixed length = l: middle loop runs  $(n - l + 1)(n-l+1)$  times, inner loop runs  $(l - 1)(l-1)$  times each.

$$T(n) = \sum_{l=2}^n (n - l + 1)(l - 1) = \frac{n^3 - n}{6}$$

So:

$$T(n) = \Theta(n^3)$$

Therefore the time complexity is the  
 $T(n) = \Theta(n^3)$

**Conclusion :-**

The Matrix Chain Multiplication problem — finding the optimal parenthesization to minimize scalar multiplications — is efficiently solved using **dynamic programming** by exploiting its two key properties: **optimal substructure** (the best solution to the full chain is built from best solutions to subchains) and **overlapping subproblems** (the same subchains recur many times across different splits).

- **Time Complexity:**  $\Theta(n^3)\Theta(n^3)$  — three nested loops over chain length, starting index, and split point.
- **Space Complexity:**  $\Theta(n^2)\Theta(n^2)$  — two tables (m for costs, s for split points) covering all  $(i, j)(i, j)$  subchain pairs.



**Marwadi**  
University  
Marwadi Chandarana Group



**FACULTY OF TECHNOLOGY**  
**Department of AI, ML & DS**  
**01AI0506- DAA Lab Manual**

---